# LEZIONE 13 - Array Monodimensionali

## PARTE TEORICA (2h)

### 1. Concetto di Array

Un **array** è una struttura dati che permette di memorizzare **più valori dello stesso tipo** in una sequenza contigua di memoria, accessibili tramite un **indice**.

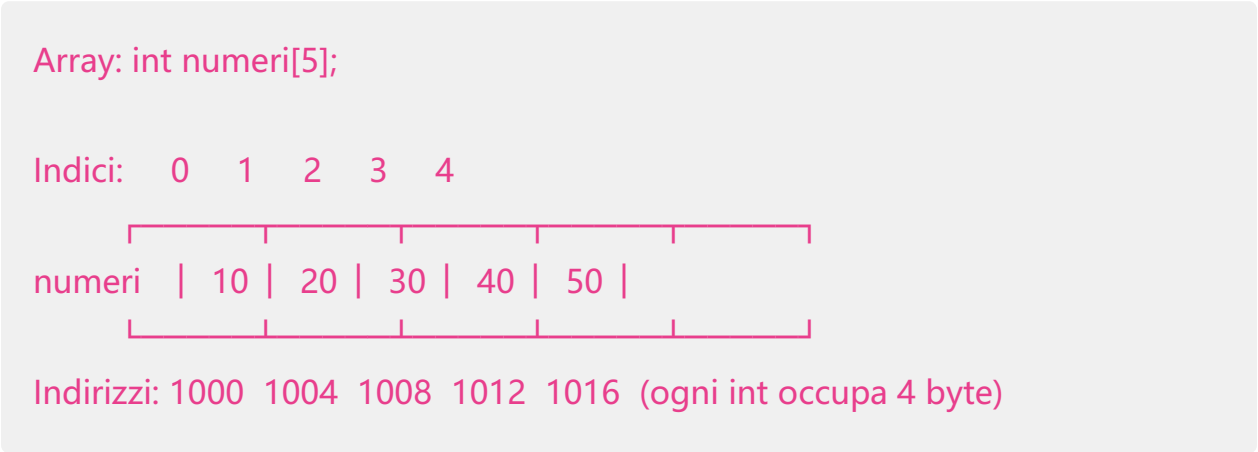
**Perché usare gli array?**

- Gestire collezioni di dati correlati (es: voti di studenti, temperature del mese)
- Evitare di dichiarare decine di variabili separate
- Facilitare l'elaborazione di grandi quantità di dati
- Permettere l'accesso rapido tramite indice

**Caratteristiche principali:**

- Tutti gli elementi hanno lo **stesso tipo** di dato
- La **dimensione è fissa** (definita alla dichiarazione)
- Gli elementi sono **contigui in memoria**
- L'accesso avviene tramite **indice** che parte da **0**

**Rappresentazione in memoria:**




---

## 2. Dichiarazione e Inizializzazione

**Sintassi base:**

```
tipo nomeArray[dimensione];
```

**Esempi di dichiarazione:**

```
int voti[30];          // Array di 30 interi
float temperature[12];  // Array di 12 float
char nome[50];         // Array di 50 caratteri
double prezzi[100];    // Array di 100 double
```

**Inizializzazione:**

```
// Metodo 1: Inizializzazione alla dichiarazione
int numeri[5] = {10, 20, 30, 40, 50};

// Metodo 2: Inizializzazione parziale (i restanti elementi sono 0)
int numeri[5] = {10, 20};  // {10, 20, 0, 0, 0}

// Metodo 3: Dimensione automatica
int numeri[] = {10, 20, 30, 40, 50};  // Dimensione = 5

// Metodo 4: Inizializzazione a zero
int numeri[5] = {0};  // Tutti gli elementi a 0

// Metodo 5: Senza inizializzazione (valori casuali!)
int numeri[5];  // ATTENZIONE: contiene valori casuali (garbage)
```

**Esempio completo:**

```
#include <stdio.h>
```

```c
int main() {
    // Dichiarazione e inizializzazione
    int numeri[5] = {10, 20, 30, 40, 50};

    // Dichiarazione senza inizializzazione
    float temperature[7];

    // Inizializzazione manuale
    temperature[0] = 18.5;
    temperature[1] = 19.2;
    temperature[2] = 21.0;
    temperature[3] = 22.5;
    temperature[4] = 20.8;
    temperature[5] = 19.5;
    temperature[6] = 18.0;

    return 0;
}
```

---

## 3. Accesso agli Elementi

L'accesso agli elementi avviene tramite l'**operatore di indicizzazione** [] utilizzando un indice da **0** a **dimensione-1**.

**Sintassi:**

```c
nomeArray[indice]
```

**Lettura e scrittura:**

```c
int numeri[5] = {10, 20, 30, 40, 50};

// Lettura
int primo = numeri[0];   // primo = 10
```

```
int terzo = numeri[2];   // terzo = 30
int ultimo = numeri[4];  // ultimo = 50

// Scrittura
numeri[0] = 100;  // Modifica il primo elemento
numeri[2] = 300;  // Modifica il terzo elemento

// Uso in espressioni
int somma = numeri[0] + numeri[1];  // somma = 100 + 20 = 120
numeri[3] = numeri[1] * 2;          // numeri[3] = 20 * 2 = 40
```

**ATTENZIONE agli indici:**

```
int array[5];

// CORRETTO: indici da 0 a 4
array[0] = 10;  // OK
array[4] = 50;  // OK

// ERRORE: indice fuori range
array[5] = 60;   // ERRORE! Non esiste array[5]
array[-1] = 10;  // ERRORE! Indice negativo
array[10] = 99;  // ERRORE! Accesso fuori dai limiti
```

**Conseguenze degli errori di indice:**

- **Comportamento indefinito** (undefined behavior)
- Possibili crash del programma
- Corruzione di dati in memoria
- Bug difficili da trovare

---

## 4. Dimensione dell'Array

**Calcolare la dimensione:**

```c
int numeri[10];

// Numero di elementi
int dimensione = sizeof(numeri) / sizeof(numeri[0]);
// sizeof(numeri) = 40 byte (10 elementi × 4 byte)
// sizeof(numeri[0]) = 4 byte (1 int)
// dimensione = 40 / 4 = 10

// Definire costanti per la dimensione
#define MAX_ELEMENTI 100

int array[MAX_ELEMENTI];
```

**Esempio pratico:**

```c
#include <stdio.h>

int main() {
    int numeri[] = {10, 20, 30, 40, 50};

    int dimensione = sizeof(numeri) / sizeof(numeri[0]);

    printf("Dimensione dell'array: %d elementi\n", dimensione);
    printf("Dimensione in byte: %lu\n", sizeof(numeri));
    printf("Dimensione di un elemento: %lu byte\n", sizeof(numeri[0]));

    return 0;
}
```

---

## 5. Array e Cicli

I cicli sono essenziali per lavorare con gli array. Il ciclo **for** è il più utilizzato.

**Pattern base:**

```c
int array[10];
int dimensione = 10;

// Inizializzazione
for (int i = 0; i < dimensione; i++) {
    array[i] = i * 10;
}

// Lettura/Stampa
for (int i = 0; i < dimensione; i++) {
    printf("%d ", array[i]);
}

// Elaborazione
for (int i = 0; i < dimensione; i++) {
    array[i] = array[i] * 2;  // Raddoppia ogni elemento
}
```

**Esempio completo: Input e Output**

```c
#include <stdio.h>

#define MAX 5

int main() {
    int numeri[MAX];

    // Input: chiede all'utente di inserire i valori
    printf("Inserisci %d numeri:\n", MAX);
    for (int i = 0; i < MAX; i++) {
        printf("Numero %d: ", i + 1);
        scanf("%d", &numeri[i]);
```

```
    }

    // Output: stampa i valori
    printf("\nNumeri inseriti:\n");
    for (int i = 0; i < MAX; i++) {
        printf("numeri[%d] = %d\n", i, numeri[i]);
    }

    return 0;
}
```

---

## 6. Operazioni Comuni sugli Array

**1. Somma di tutti gli elementi:**

```
int somma = 0;
for (int i = 0; i < dimensione; i++) {
    somma += array[i];
}
```

**2. Media:**

```
float media = (float)somma / dimensione;
```

**3. Ricerca del massimo:**

```
int massimo = array[0];
for (int i = 1; i < dimensione; i++) {
    if (array[i] > massimo) {
        massimo = array[i];
    }
}
```

**4. Ricerca del minimo:**

```
int minimo = array[0];
for (int i = 1; i < dimensione; i++) {
    if (array[i] < minimo) {
        minimo = array[i];
    }
}
```

**5. Ricerca di un elemento:**

```
int target = 30;
int trovato = 0;
int posizione = -1;

for (int i = 0; i < dimensione; i++) {
    if (array[i] == target) {
        trovato = 1;
        posizione = i;
        break;
    }
}

if (trovato) {
    printf("Elemento trovato in posizione %d\n", posizione);
} else {
    printf("Elemento non trovato\n");
}
```

**6. Conteggio elementi che soddisfano una condizione:**

```
// Conta numeri pari
int contatore = 0;
for (int i = 0; i < dimensione; i++) {
```

```
    if (array[i] % 2 == 0) {
        contatore++;
    }
}
```

---

# PARTE PRATICA (2h)

## Esercizio 1: Inserimento e stampa di array

```c
#include <stdio.h>

#define MAX 10

int main() {
    int numeri[MAX];
    int n;

    // Chiedi quanti elementi inserire
    do {
        printf("Quanti numeri vuoi inserire (max %d)? ", MAX);
        scanf("%d", &n);
    } while (n < 1 || n > MAX);

    // Inserimento
    printf("\nInserisci %d numeri:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Numero %d: ", i + 1);
        scanf("%d", &numeri[i]);
    }

    // Stampa normale
    printf("\nArray in ordine normale:\n");
```

```c
    for (int i = 0; i < n; i++) {
        printf("%d ", numeri[i]);
    }
    printf("\n");

    // Stampa inversa
    printf("\nArray in ordine inverso:\n");
    for (int i = n - 1; i >= 0; i--) {
        printf("%d ", numeri[i]);
    }
    printf("\n");

    // Stampa con indici
    printf("\nArray con indici:\n");
    for (int i = 0; i < n; i++) {
        printf("[%d] = %d\n", i, numeri[i]);
    }

    return 0;
}
```

---

## Esercizio 2: Ricerca di elementi

```c
#include <stdio.h>

#define MAX 10

int main() {
    int numeri[MAX] = {15, 8, 23, 42, 7, 19, 31, 5, 12, 28};
    int dimensione = MAX;
    int target;

    // Stampa array
```

```c
    printf("Array: ");
    for (int i = 0; i < dimensione; i++) {
        printf("%d ", numeri[i]);
    }
    printf("\n\n");

    // Ricerca elemento
    printf("Inserisci il numero da cercare: ");
    scanf("%d", &target);

    // Ricerca lineare
    int trovato = 0;
    int posizioni[MAX];
    int contatore = 0;

    for (int i = 0; i < dimensione; i++) {
        if (numeri[i] == target) {
            posizioni[contatore] = i;
            contatore++;
            trovato = 1;
        }
    }

    // Risultato
    if (trovato) {
        printf("\nElemento %d trovato in %d posizione/i:\n", target, contatore);
        for (int i = 0; i < contatore; i++) {
            printf("  - Indice %d\n", posizioni[i]);
        }
    } else {
        printf("\nElemento %d non trovato nell'array\n", target);
    }

    // Verifica se un numero è presente
    printf("\nVerifichiamo la presenza di alcuni numeri:\n");
```

```
    int daVerificare[] = {7, 100, 23, 50};

    for (int i = 0; i < 4; i++) {
        int presente = 0;
        for (int j = 0; j < dimensione; j++) {
            if (numeri[j] == daVerificare[i]) {
                presente = 1;
                break;
            }
        }
        printf("%d: %s\n", daVerificare[i], presente ? "PRESENTE" : "ASSENTE");
    }

    return 0;
}
```

---

## Esercizio 3: Calcolo di media, massimo, minimo

```
#include <stdio.h>

#define MAX 100

int main() {
    float numeri[MAX];
    int n;

    // Input
    printf("Quanti numeri vuoi inserire? ");
    scanf("%d", &n);

    printf("\nInserisci %d numeri:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Numero %d: ", i + 1);
```

```
    scanf("%f", &numeri[i]);
}

// Calcolo somma
float somma = 0;
for (int i = 0; i < n; i++) {
    somma += numeri[i];
}

// Calcolo media
float media = somma / n;

// Ricerca massimo e minimo
float massimo = numeri[0];
float minimo = numeri[0];
int indiceMassimo = 0;
int indiceMinimo = 0;

for (int i = 1; i < n; i++) {
    if (numeri[i] > massimo) {
        massimo = numeri[i];
        indiceMassimo = i;
    }
    if (numeri[i] < minimo) {
        minimo = numeri[i];
        indiceMinimo = i;
    }
}

// Calcolo range
float range = massimo - minimo;

// Conteggio sopra/sotto media
int sopraMadia = 0, sottoMedia = 0, ugualeMedia = 0;
for (int i = 0; i < n; i++) {
```

```
        if (numeri[i] > media) sopraMadia++;
        else if (numeri[i] < media) sottoMedia++;
        else ugualeMedia++;
    }

    // Output statistiche
    printf("\n=== STATISTICHE ===\n");
    printf("Numero di elementi: %d\n", n);
    printf("Somma: %.2f\n", somma);
    printf("Media: %.2f\n", media);
    printf("Massimo: %.2f (posizione %d)\n", massimo, indiceMassimo);
    printf("Minimo: %.2f (posizione %d)\n", minimo, indiceMinimo);
    printf("Range: %.2f\n", range);
    printf("\nDistribuzione rispetto alla media:\n");
    printf("  Sopra la media: %d (%.1f%%)\n", sopraMadia, (float)sopraMadia/
           n*100);
    printf("  Sotto la media: %d (%.1f%%)\n", sottoMedia, (float)sottoMedia/
           n*100);
    printf("  Uguale alla media: %d (%.1f%%)\n", ugualeMedia, (float)ugualeM
           edia/n*100);

    return 0;
}
```

---

## Esercizio 4: Inversione di un array

```
#include <stdio.h>

#define MAX 20

void stampaArray(int arr[], int n, const char* messaggio) {
    printf("%s: ", messaggio);
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
```

```
    }
    printf("\n");
}

int main() {
    int numeri[MAX];
    int n;

    // Input
    printf("Quanti numeri vuoi inserire? ");
    scanf("%d", &n);

    printf("Inserisci %d numeri:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &numeri[i]);
    }

    // Stampa array originale
    stampaArray(numeri, n, "Array originale");

    // Metodo 1: Inversione con array temporaneo
    int temp[MAX];
    for (int i = 0; i < n; i++) {
        temp[i] = numeri[n - 1 - i];
    }
    for (int i = 0; i < n; i++) {
        numeri[i] = temp[i];
    }
    stampaArray(numeri, n, "Dopo inversione (metodo 1)");

    // Rimettiamo l'array come prima (per testare metodo 2)
    for (int i = 0; i < n; i++) {
        temp[i] = numeri[n - 1 - i];
    }
    for (int i = 0; i < n; i++) {
```

```
        numeri[i] = temp[i];
    }

    // Metodo 2: Inversione in place (scambio elementi)
    printf("\nMetodo 2: Inversione in place\n");
    for (int i = 0; i < n / 2; i++) {
        // Scambia numeri[i] con numeri[n-1-i]
        int temporaneo = numeri[i];
        numeri[i] = numeri[n - 1 - i];
        numeri[n - 1 - i] = temporaneo;

        // Mostra lo scambio
        printf("Scambio %d: ", i + 1);
        stampaArray(numeri, n, "");
    }

    stampaArray(numeri, n, "\nArray finale invertito");

    return 0;
}
```

---

## Esercizio 5: Operazioni avanzate su array

```
#include <stdio.h>

#define MAX 50

int main() {
    int numeri[MAX] = {12, 7, 23, 7, 45, 12, 89, 7, 23, 56};
    int n = 10;

    printf("Array originale: ");
    for (int i = 0; i < n; i++) {
```

```
    printf("%d ", numeri[i]);
}
printf("\n\n");

// 1. Conta elementi pari e dispari
int pari = 0, dispari = 0;
for (int i = 0; i < n; i++) {
    if (numeri[i] % 2 == 0) pari++;
    else dispari++;
}
printf("Numeri pari: %d, Numeri dispari: %d\n\n", pari, dispari);

// 2. Trova duplicati
printf("Elementi duplicati:\n");
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (numeri[i] == numeri[j]) {
            // Verifica che non l'abbiamo già stampato
            int giàStampato = 0;
            for (int k = 0; k < i; k++) {
                if (numeri[k] == numeri[i]) {
                    giàStampato = 1;
                    break;
                }
            }
            if (!giàStampato) {
                printf("  %d appare più volte\n", numeri[i]);
            }
            break;
        }
    }
}

// 3. Frequenza degli elementi
printf("\nFrequenza degli elementi:\n");
```

```
    int visitato[MAX] = {0};
    for (int i = 0; i < n; i++) {
        if (visitato[i] == 0) {
            int conteggio = 1;
            for (int j = i + 1; j < n; j++) {
                if (numeri[i] == numeri[j]) {
                    conteggio++;
                    visitato[j] = 1;
                }
            }
            printf("  %d appare %d volta/e\n", numeri[i], conteggio);
        }
    }

    // 4. Secondo elemento più grande
    int max1 = numeri[0], max2 = -1;
    for (int i = 1; i < n; i++) {
        if (numeri[i] > max1) {
            max2 = max1;
            max1 = numeri[i];
        } else if (numeri[i] > max2 && numeri[i] != max1) {
            max2 = numeri[i];
        }
    }
    printf("\nMassimo: %d, Secondo massimo: %d\n", max1, max2);

    // 5. Copia elementi pari in un nuovo array
    int arrayPari[MAX];
    int contatorePari = 0;
    for (int i = 0; i < n; i++) {
        if (numeri[i] % 2 == 0) {
            arrayPari[contatorePari] = numeri[i];
            contatorePari++;
        }
    }
```

```
    printf("\nArray contenente solo numeri pari:\n");
    for (int i = 0; i < contatorePari; i++) {
        printf("%d ", arrayPari[i]);
    }
    printf("\n");

    return 0;
}
```

---

## Esercizi da svolgere autonomamente:

1. **Ordinamento Bubble Sort**: Implementa l'algoritmo bubble sort per ordinare un array in ordine crescente
2. **Rotazione array**: Ruota gli elementi di un array verso destra di K posizioni (es: [1,2,3,4,5] ruotato di 2 diventa [4,5,1,2,3])
3. **Merge di due array**: Dati due array ordinati, crea un terzo array contenente tutti gli elementi in ordine
4. **Rimozione duplicati**: Rimuovi tutti gli elementi duplicati da un array mantenendo solo la prima occorrenza
5. **Array palindromo**: Verifica se un array è palindromo (si legge uguale da sinistra a destra e viceversa)
6. **Differenza tra array**: Dati due array, trova gli elementi presenti nel primo ma non nel secondo
7. **Prodotto scalare**: Calcola il prodotto scalare di due array (somma dei prodotti degli elementi corrispondenti)
8. **Elementi unici**: Crea un nuovo array contenente solo gli elementi che appaiono una sola volta
9. **Shift ciclico**: Implementa uno shift ciclico a sinistra (il primo elemento va alla fine)

10. **Array di Fibonacci**: Riempi un array con i primi N numeri della sequenza di Fibonacci

---

# Riepilogo Concetti Chiave

✓ Gli **array** sono collezioni di elementi dello stesso tipo
✓ Gli indici vanno da **0** a **dimensione-1**
✓ Accesso fuori dai limiti causa **comportamento indefinito**
✓ I **cicli for** sono perfetti per iterare sugli array
✓ Operazioni comuni: ricerca, massimo/minimo, somma, media
✓ `sizeof(array)/sizeof(array[0])` calcola il numero di elementi

**Prossima lezione**: Approfondiremo gli **array multidimensionali** e le **stringhe**!